SRM Institute of Science and Technology
College of Engineering and Technology
School of Computing
SRM Nagar, Kattankulathur – 603203, Chengalpattu District, Tamilnadu
Academic Year: 2025-26 (ODD)

SET D

BATCH 2

Test: FJ - II | **Date: 26/09/2025**
Course Code & Title: 21CSC201J Data structures and Algorithm | **Duration:** 1 hour 40 min
Year & Sem: II Year / III Sem | **Max. Marks:** 50

Course Articulation Matrix:

Q. No	Question
1.	A singly linked list contains the elements:5 → 15 → 25 → 35 → 45 → NULL What will be the output of the following pseudocode? ptr ← head while ptr ≠ NULL do print(ptr.data) ptr ← ptr.next.next end while A. 5 15 25 35 45 B. 5 25 45 C. 45 25 5 D. 5 15 35 **Ans: B. 5 25 45**
2.	In a Josephus problem with n = 10 people standing in a circle and every fourth person (k=4) is eliminated, what is the position of the last survivor (1-based indexing)? **Answer:** The last survivor is in **position 5**
3.	Suppose a stack has a maximum size of 4. The following operations are performed: PUSH(10), PUSH(20), POP(), PUSH(30), PUSH(40), PUSH(50) Which elements are finally in the stack (from bottom to top)? A. 10, 30, 40, 50 B. 20, 30, 40, 50 C. 10, 30, 40 D. 10, 20, 30, 40 **Ans: A 10, 30, 40, 50**
4.	What is the postfix expression of the following infix expression: A + ((B + C) * ((D + E) ^ F)) **Ans: A B C + D E + F ^ * +**

PART B

5	Consider the single linked list is created and head is given. Give an algorithm to find

middle node of the linked list with the following conditions.

a) If the number of nodes is odd, return the middle node.
b) If the number of nodes is even, there are two middle nodes, so return the second middle node

Answer

```
function findMiddle(head):
  if head == NULL:
    return NULL

  # Step 1: Count nodes
  count ← 0
  temp ← head
  while temp ≠ NULL:
    count ← count + 1
    temp ← temp.next

  # Step 2: Find middle index (second middle for even)
  middleIndex ← (count // 2) + 1

  # Step 3: Traverse to middle node
  temp ← head
  for i = 1 to middleIndex - 1:
    temp ← temp.next

  return temp
```

6 A hospital emergency room manages patients using a data structure based on the severity of their condition (1 = most critical, 5 = least critical). Patients arrive in the following order:

Patient	**Severity**
P1	3
P2	1
P3	4
P4	2
P5	1

Suggest a suitable Data Structure and represent pictorially of showing the order in which they will be treated. Give the Pseudocode for the insertion process.

Answer

Order of Execution

Step	**Operation**	**Queue State (front = highest priority)**
Initialization	Create empty queue	[ ]
Arrival: P1	Insert P1 (3)	[P1(3)]
Arrival: P2	Insert P2 (1)	[P2(1), P1(3)]
Arrival: P3	Insert P3 (4)	[P2(1), P1(3), P3(4)]

Arrival: P4	Insert P4 (2)	[P2(1), P4(2), P1(3), P3(4)]
Arrival: P5	Insert P5 (1)	[P2(1), P5(1), P4(2), P1(3), P3(4)]

Step-by-Step Explanation
When a patient arrives, they are inserted into the queue according to severity: smaller severity value (higher priority) comes earlier in the queue.

After all insertions, the queue order (front to back) for treatment is: P2(1), P5(1), P4(2), P1(3), P3(4).

Treating Patients (Remove Operations)
First treated: Remove P2, queue becomes [P5(1), P4(2), P1(3), P3(4)].
Next: Remove P5, queue becomes [P4(2), P1(3), P3(4)].
Next: Remove P4, queue becomes [P1(3), P3(4)].
Next: Remove P1, queue becomes [P3(4)].
Last: Remove P3, queue becomes [] (empty).

Pseudocode

```
function insertPatient(queue, patient, severity):
   newNode = Node(patient, severity)
   // If queue is empty or new patient has higher priority than front
   if queue.head == null or severity < queue.head.severity:
      newNode.next = queue.head
      queue.head = newNode
      return
   // Traverse to find correct position
   current = queue.head
   while current.next != null and current.next.severity <= severity:
      current = current.next
   newNode.next = current.next
   current.next = newNode
```

7 A database system stores records that need to be frequently accessed, inserted, and deleted. Suppose a database contains n records,
Write a Pseudocode to:

a) Insert after the middle element if the database is stored as an Array, (**2 Marks**)
b) Access the middle element if the database is stored as a Singly Linked List (**2 Marks**)
c) Delete the middle element if the database is stored as a Doubly Linked List (**2 Marks**)
d) State their time complexities. (**2 Marks**)

Answer:

Insert After the Middle Element (Array)
Pseudocode:

```
function insertAfterMiddle(arr, n, value):
   mid = n // 2  // For even n, this is the lower middle
   // Shift elements right from end to mid+1
   for i = n to mid+1 step -1:
      arr[i] = arr[i-1]
   arr[mid+1] = value
```

n = n + 1

Access the Middle Element (Singly Linked List)

Pseudocode:

```
function accessMiddle(head, n):
    mid = n // 2
    current = head
    for i = 0 to mid-1:
        current = current.next
    return current.data
```

Delete the Middle Element (Doubly Linked List)

Pseudocode:

```
function deleteMiddle(head, n):
    mid = n // 2
    current = head
    for i = 0 to mid-1:
        current = current.next
    // Remove current node
    if current.prev != null:
        current.prev.next = current.next
    if current.next != null:
        current.next.prev = current.prev
    // If current is head, update head
    if current == head:
        head = current.next
    n = n – 1
```

Operation	Data Structure	Time Complexity
Insert after middle	Array	O(n)
Access middle	Singly Linked List	O(n)
Delete middle	Doubly Linked List	O(n)

8 Given a singly linked list and a positive integer k, design an algorithm to reverse the list in groups of k nodes. If the remaining nodes are fewer than k, leave them unchanged.

Explain the algorithm step-by-step, trace pointer movements for each group, and illustrate your approach with the example. Discuss the time and space complexity of your solution.

Hint: Input list: 10 → 20 → 30 → 40 → 50 → 60 → NULL with k=3

Expected output: 30 → 20 → 10 → 60 → 50 → 40 → NULL

Answer:

```
function reverseKGroup(head, k):
    if head is null or k == 1:
        return head

    dummy = new Node(0)
    dummy.next = head
    prev_tail = dummy
    current = head
```

```
  while current is not null:
    # Check if there are at least k nodes left
    count = 0
    node = current
    while node is not null and count < k:
      node = node.next
      count += 1
    if count < k:
      break  # Fewer than k nodes left; do not reverse

    # Reverse k nodes
    prev = null
    tail = current
    for i from 0 to k-1:
      next_node = current.next
      current.next = prev
      prev = current
      current = next_node

    # Connect previous group to reversed group
    prev_tail.next = prev
    tail.next = current
    prev_tail = tail

  return dummy.next
```

Example:

Input:10 → 20 → 30 → 40 → 50 → 60 → NULL, k=3

First group (10, 20, 30):

- current` starts at 10.
- Check 3 nodes present? Yes: 10, 20, 30.
- Reverse these 3:
  - 10.next = null (after reversal, it becomes tail).
  - 20.next = 10
  - 30.next = 20
- Reversed group: 30 → 20 → 10
- Set new_head= 30
- Set prev_tail = 10 (tail of reversed group)
- Move current to 40 (node after 10)

Second group (40, 50, 60):

- current at 40.
- Check 3 nodes present? Yes: 40, 50, 60.

	- Reverse these 3: - 40.next = null (tail after reversal) - 50.next = 40 - 60.next = 50 - Reversed group: 60 → 50 → 40 - Link previous tail (10.next) to 60 - Update prev_tail = 40 - Move current to null (end of list) No more nodes left. Final output linked list: 30 → 20 → 10 → 60 → 50 → 40 → NULL **Time and Space Complexity** - Time complexity: O(n) where n is the number of nodes in the linked list. Each node is visited and reversed at most once. - Space complexity: O(1) as the reversal is done in-place using constant extra space.
9	Consider the following sparse matrix A of size 5 × 5: $\begin{bmatrix} 0 & 0 & 8 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 5 & 0 & 0 & 0 & 3 \\ 0 & 9 & 0 & 0 & 0 \\ 0 & 0 & 0 & 7 & 0 \end{bmatrix}$ (a) Write the complete triplet representation of matrix S, including the header. (**4 Marks**) (b) Write pseudocode for a function SparseToTriplet(mat, rows, cols) that: • Accepts a 2D array mat of size rows × cols, • Scans the matrix in row-major order, Constructs and returns the triplet representation as a list of tuples. (**4 Marks)** **a) Triplet Representation** **Row** **Col** **Value** 5 5 5 0 2 8 2 0 5 2 4 3 3 1 9 4 3 7 **b)** Pseudocode function SparseToTriplet(mat, rows, cols):

```
triplet_list = []
count = 0
for i from 0 to rows-1:
   for j from 0 to cols-1:
      if mat[i][j] != 0:
         triplet_list.append((i, j, mat[i][j]))
         count = count + 1
# Insert header at the beginning
triplet_list.insert(0, (rows, cols, count))
return triplet_list
```

10 You are given a string containing different types of brackets (), and [].
Design an algorithm that takes as input a sequence of symbols from { (,), [,] } using a stack to validate if the sequence forms a balanced parenthesis sequence.
When will you declare the sequence as not balanced?
What would be the time complexity?

Algorithm

- Initialize an empty stack.
- Scan the string from left to right:
  - If you see an opening bracket ((or [), push it onto the stack.
  - If you see a closing bracket () or]):
    - If the stack is empty, the sequence is not balanced (there's nothing to match the closing bracket).
    - Otherwise, pop the top of the stack and check:
      - If the popped bracket is not the matching opening bracket (e.g., you see) but popped [), the sequence is not balanced.

After scanning the string:

- If the stack is not empty, the sequence is not balanced (there are unmatched opening brackets).
- If the stack is empty, the sequence is balanced.

When is the sequence not balanced?

- encounter a closing bracket but the stack is empty (no opening to match).
- pop from the stack and the bracket does not match the closing bracket.
- After processing, the stack is not empty (leftover openings).

Time Complexity

O(n), where n is the length of the string. Each character is processed once, and stack operations are O(1)

11 Evaluate the following postfix expression using a stack.
Postfix Expression:
9 3 2 * 1 + - 6 2 / +

a. Show the contents of the stack after processing each token, clearly indicating the top of the stack.(**4 Marks**)
b. Write pseudocode that evaluates any valid postfix arithmetic expression.(**4 Marks**)

a) Final result: 5 (top of the stack)

Token	Stack after processing (bottom → top)
9	9
3	9, 3
2	9, 3, 2
*	9, 6 (3 * 2 = 6)
1	9, 6, 1
+	9, 7 (6 + 1 = 7)
-	2 (9 - 7 = 2)
6	2, 6
2	2, 6, 2
/	2, 3 (6 / 2 = 3)
+	5 (2 + 3 = 5)

```
function evaluatePostfix(tokens):
    stack = empty stack
    for each token in tokens:
        if token is a number:
            stack.push(token)
        else if token is an operator:
            operand2 = stack.pop()
            operand1 = stack.pop()
            result = apply_operator(operand1, operand2, token)
            stack.push(result)
    return stack.pop()

function apply_operator(a, b, op):
    if op == '+': return a + b
    if op == '-': return a - b
    if op == '*': return a * b
    if op == '/': return a / b  # or integer division if required
```

12 You are given a sequence of numbers representing daily stock prices. Design an algorithm using a stack to find the "next greater element" for each element in the sequence. The next greater element for an element x is the first element to its right that is greater than x. If no such element exists, consider it -1

Sample Test Case

```
arr = [13, 7, 6, 12]
#  Output : [-1, 12, 12, -1]
arr = [1, 2, 3, 4]
#  Output : [2, 3, 4, -1]
```

Answer

1. Initialize an empty stack and a result array of size n (all values set to −1).
2. Traverse the array from right to left:

	For each element arr[i]: • While the stack is not empty and the top of the stack<= arr[i], pop the stack • If the stack is not empty after popping, the top of the stack is the next greater element for arr[i] • Otherwise, the next greater element is -1 • Push arr[i] onto the stack 3. Return the result array.